PROJECT PORTFOLIO

AI Engineering & Consultation — Selected Work

Three projects demonstrating end-to-end AI system design: from RAG pipelines and LLM agent infrastructure to applied data engineering. Built with Python, LangGraph, MCP, FastAPI, and Claude.

Python · AI/ML · LLM Agents August 2026

Project 1

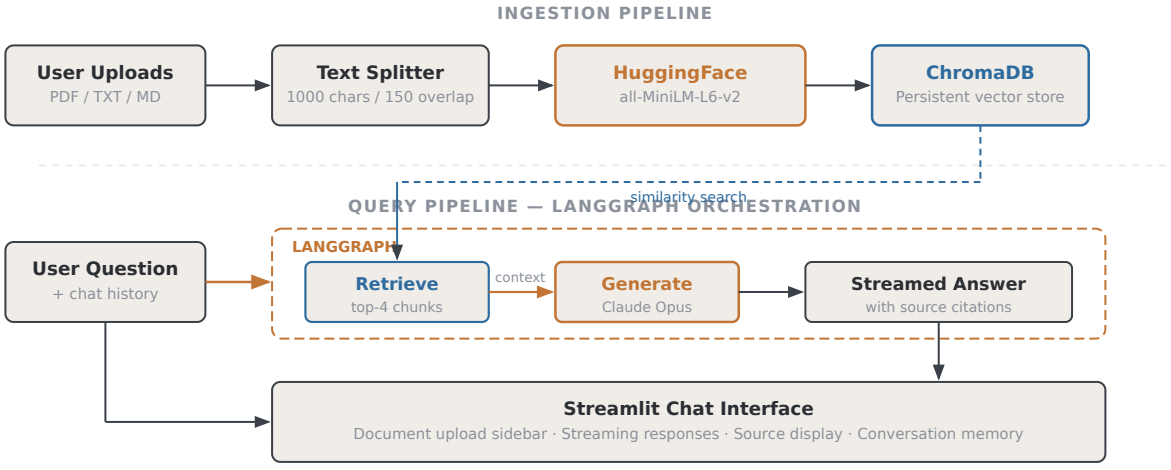
RAG Chatbot — Document-Grounded Q&A System

A retrieval-augmented generation chatbot that lets users upload documents and ask questions, receiving answers grounded in those documents with inline source citations.

TECHNOLOGY STACK

`LangGraph` `Claude API` `ChromaDB` `HuggingFace Embeddings` `Streamlit` `LangChain` `Python asyncio`

ARCHITECTURE WORKFLOW



Two-stage RAG pipeline: offline document ingestion with local embeddings, and LangGraph-orchestrated query-time retrieval + generation.

WHAT I BUILT

- Designed and implemented a complete RAG pipeline with LangGraph's `StateGraph` for orchestrating retrieve → generate flow
- Used local HuggingFace sentence-transformers for embeddings — indexing and retrieval run fully offline with zero API cost
- Persistent ChromaDB vector store survives restarts; supports PDF, TXT, and Markdown document ingestion
- Streaming answer generation via Claude with inline source citations from retrieved passages
- Built conversation memory (last 6 turns) for contextual follow-up questions

- Provider-agnostic architecture — embeddings can be swapped to Voyage AI or OpenAI with a single function change

KEY METRICS

2-stage	Top-4	$0	3 formats
LangGraph pipeline	Chunks per retrieval	Embedding cost (local)	PDF / TXT / MD

Project 2

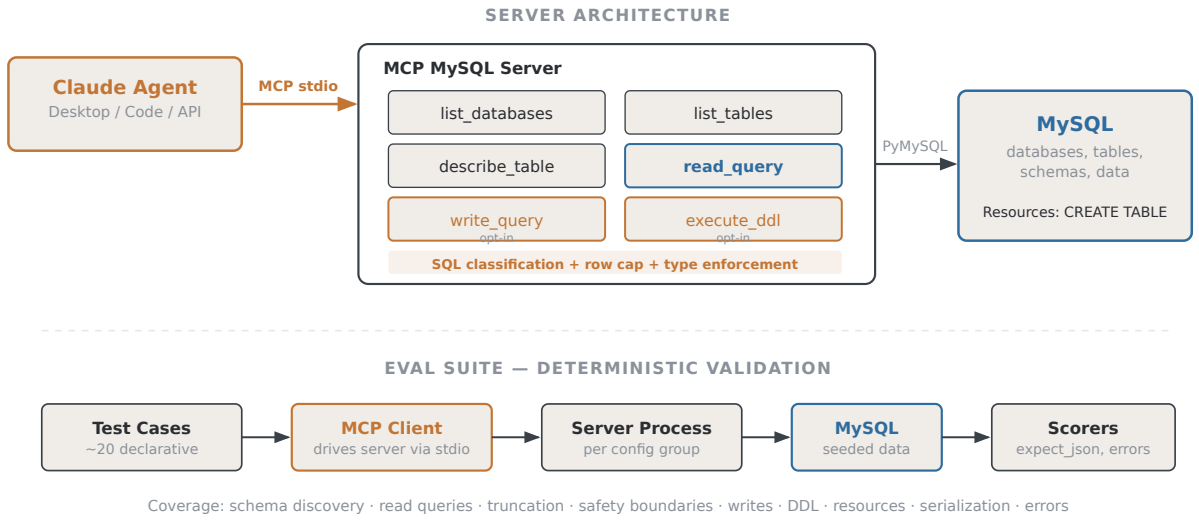
MCP MySQL Server — AI Agent Database Infrastructure

A Model Context Protocol server that gives AI agents (Claude Desktop, Claude Code) structured, safe access to MySQL databases — with a companion eval suite for deterministic validation.

TECHNOLOGY STACK

`MCP SDK` `Claude Agents` `PyMySQL` `Python asyncio` `SQLAlchemy` `pytest` `dotenv`

ARCHITECTURE WORKFLOW



MCP server exposes 6 tools with safety-first defaults; the eval suite drives the server end-to-end via a real MCP client session.

WHAT I BUILT

- Built a production MCP server exposing 6 tools for database interaction — reads enabled by default, writes and DDL explicitly opt-in via environment flags
- Implemented SQL statement classification that rejects mismatched operations (read_query won't accept INSERT, write_query won't accept DDL)
- Built a companion deterministic eval suite that launches the server as a subprocess, drives it through real MCP client sessions, and validates across 9 test categories
- Designed composable scorer functions (`expect_json`, `expect_error`, `values_contain`) for declarative test case definitions

- Published as both a Claude Desktop skill (zip upload) and a Claude Code plugin (GitHub-linked, auto-updating)

KEY METRICS

6 tools	23/23	9 categories	Read-only
MCP-exposed operations	Eval pass rate	Test coverage areas	Default safety mode

Project 3

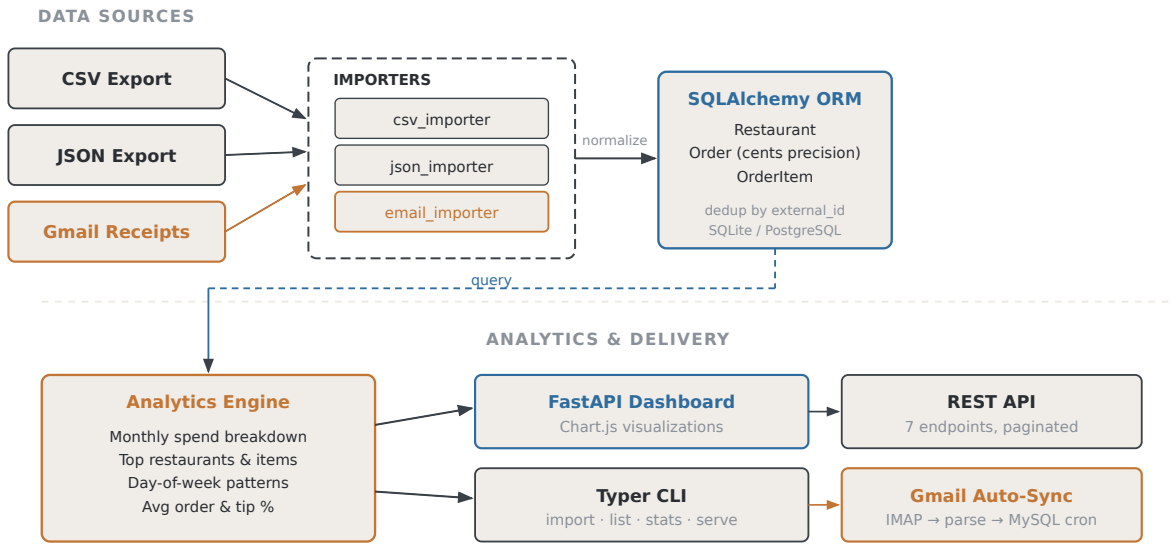
Uber Eats Tracker — Personal Spend Analytics Platform

A data engineering project that imports Uber Eats order data from multiple sources (CSV, JSON, receipt emails), stores it in a normalized database, and surfaces spending analytics through a web dashboard and CLI.

TECHNOLOGY STACK

FastAPI · SQLAlchemy 2.0 · Chart.js · Typer CLI · PyMySQL · IMAP / email parsing · pytest

ARCHITECTURE WORKFLOW



Multi-source ETL pipeline: three importers normalize data into a cents-precision ORM; analytics engine serves insights via dashboard, API, and CLI.

WHAT I BUILT

- Built three pluggable importers (CSV, JSON, .eml email) with a shared normalization layer and cents-precision money storage to avoid float drift
- Designed the email importer with regex-based receipt parsing tolerant of formatting changes, with clear error boundaries
- Implemented a Gmail auto-sync script using IMAP that fetches new Uber Eats receipts since the last known order date and inserts deduplicated records
- Built 5 analytics functions: summary stats, monthly breakdown, top restaurants, top items, day-of-week spending patterns

- Delivered through both a FastAPI web dashboard (with Chart.js visualizations) and a Typer CLI for scripting and automation
- Full test suite with in-memory SQLite for fast, isolated tests

KEY METRICS

3 importers	7 endpoints	5 analytics	Auto-sync
CSV / JSON / Email	REST API	Spend insights	Gmail IMAP → MySQL

About this portfolio. These projects are open-source and available on GitHub. Each was designed, built, and tested end-to-end — from architecture decisions through implementation and deployment. Code samples and full repositories available upon request.

Core competencies demonstrated: RAG pipeline design, LLM agent infrastructure (MCP), vector databases, data engineering & ETL, API design, evaluation frameworks, full-stack development, Python ecosystem fluency.